

Passing Is Not Shipping

Correctness, Semantic Equivalence and ReDoS Safety in
LLM-Generated Regular Expressions, and the Limits of Measuring Them

Foothills Labs
info@foothills-labs.com

August 2026

Abstract

We evaluate eleven contemporary language models on natural-language-to-regex generation over 450 tasks with three samples each (14,850 requests, \$10.95), scoring every answer on three axes: whether it satisfies the task’s tests, whether it denotes the same language as a human reference, and whether it is vulnerable to regular-expression denial of service (ReDoS).

Our headline result is a negative one about a composite metric we constructed and then had to abandon. A composite requiring correctness, safety and semantic equivalence appears to show that “passing is twice as easy as shipping”: models pass 38.0–47.5% of tasks but satisfy all three criteria on only 17.1–24.8%. Decomposing that gap shows that **87% of it is contributed by the semantic-equivalence term**, and a manual audit shows that term is itself dominated by reference-set error and task underspecification rather than model behaviour. Using only the two criteria that never consult a human answer key, the gap is **2.9 percentage points**: just 7.4% of functionally correct patterns are ReDoS-vulnerable.

That figure is itself informative by contrast. Joint correctness-and-security benchmarks in general code generation report far larger penalties. BaxBench finds roughly half of correct backends exploitable. SecureAgentBench reports 15.2% correct-and-secure for its best agent. The regex domain does *not* reproduce that pattern, and we report the difference rather than the similarity. Separately, *models are safer than the humans whose answers define the benchmark*: 13.6% of the corpus’s human-written reference patterns are ReDoS-vulnerable against 9.0% for models pooled, and every individual model is safer than the reference set.

The audit behind this is the paper’s methodological contribution. Metrics that compare generated output against a human answer key inherit that key’s error rate. In a random sample of disagreements where a model passed every test yet was scored as semantically different, the model was clearly at fault in roughly 15% of cases. The rest split between demonstrably incorrect reference patterns (including a reference character class containing a literal `l`) and prompts that never specified the property in dispute. Because that unreliable term supplies 87% of our composite’s headline gap, the composite was measuring its own weakest component. We also show that independent binomial intervals, the default in this literature, are the wrong inferential tool when all systems answer identical items, and that a paired bootstrap resolves nine times as many pairwise comparisons on the same data.

We therefore decline to publish a ranking. We release all raw generations, scoring code and adjudications, and recommend that evaluations separate reference-dependent from reference-independent metrics and assign them different evidentiary weight.

Artifacts. All model outputs, scoring code, per-case adjudications and analysis scripts are available at <https://github.com/foothills-labs/regexleaderboard>. Every number in this paper recomputes offline from committed data with no API access.

1 Introduction

Regular expressions make an unusually good probe for language-model evaluation. The task is compact. The output runs. Correctness is partly decidable, which is rare. And a syntactically valid, test-passing answer can still be a security defect, which is what drew us to the task in the first place.

The standard protocol for this task scores a candidate against a reference by DFA equivalence [Locascio et al., 2016, Kushman and Barzilay, 2013], sometimes alongside test-based correctness. Neither criterion notices catastrophic backtracking. A pattern can satisfy every provided example, denote exactly the reference language, and still admit inputs on which matching takes time exponential in the input. That is the ReDoS vulnerability class [Davis et al., 2018], a denial-of-service vector in deployed systems and common enough in human-written code to have prompted ecosystem-scale study.

1.1 Contributions

1. **A three-axis evaluation of eleven current models** (Section 4), reporting test correctness, semantic equivalence and ReDoS safety jointly, together with a composite (USABLE) requiring all three. Joint correctness-and-security scoring is established prior art (Section 2.1). What we add is its transfer to natural-language-to-regex generation, a setting that differs from existing joint benchmarks in three respects: the vulnerability class is ReDoS rather than CWE-classified defects, correctness is *partly decidable* rather than test-only, and a per-task human reference exists, which enables Section 4.4.
2. **A human baseline for ReDoS** (Section 4.4). We apply the identical safety screen to the corpus’s own reference patterns. Models are uniformly safer than the reference set, which relocates the finding from a claim about model deficiency to a claim about the distribution models were trained on.
3. **A quantified audit of reference-standard error** (Section 5.1). We sample disagreements and adjudicate them, finding that the majority of apparent semantic errors are not model errors. Existing joint benchmarks are largely reference-free. They execute tests and exploits instead of comparing against a gold artifact, so the failure mode does not arise for them. It arises here because the NL-to-regex literature is built on reference comparison.
4. **A methodological correction** (Section 5.2). We show that independent intervals substantially understate resolvable differences when all systems answer identical items, and give the paired alternative.
5. **Full artifact release** including raw generations, which permits any of our scoring decisions to be overturned without re-running inference.

1.2 What this paper does not claim

We do not publish a ranking of the eleven models. Section 5.3 shows that 62% of tasks yield identical outcomes across all eleven systems, and that only nine of fifty-five pairwise comparisons resolve at 95% confidence even under the correct paired test. Reporting an ordinal leaderboard from this data would be unsupported by it.

We also do not claim novelty for DFA-equivalence checking, which is standard in this literature [Locascio et al., 2016], nor for the scoring engine, which is prior work by the present authors released separately [Foothills Labs, 2026] and used here as a pinned dependency.

2 Background and Related Work

NL-to-regex generation. The task was established by Kushman and Barzilay [2013] (KB13) and extended by Locascio et al. [2016] (NL-RX), both of which adopt DFA equivalence against a reference as the correctness criterion. That is the right choice in principle. $[0-9]^+$ and $[0-9][0-9]^*$ denote the same language and differ as strings, so text comparison would mark one of them wrong. But the criterion inherits any defect in the reference set, which is the dependency this paper quantifies.

ReDoS. Catastrophic backtracking arises when a backtracking matcher can decompose a prefix in exponentially many ways, canonically via a quantifier applied to a quantified group, as in $(a^+)^+$. Davis et al. [2018] document its prevalence at ecosystem scale, and Davis et al. [2019] study regex portability and re-use across languages. ICPC 2024 authors — VERIFY [2024] examine ReDoS specifically in LLM-generated patterns and report a skew toward polynomial over exponential vulnerability. We compare against that finding in Section 4.4.

2.1 Joint correctness-and-security benchmarks

Evaluating functional correctness and security together is an active area, and this paper is not its first instance. Peng et al. [2025] introduce CWEval, scoring 119 security-critical tasks across 31 CWEs and five languages with outcome-driven oracles for both properties, and report a substantial population of functionally correct but insecure generations. Vero et al. [2025] present BaxBench, 392 backend tasks with expert exploits, and find that roughly half of functionally correct solutions are exploitable. Chen et al. [2025] evaluate coding *agents* on 105 repository-level tasks, reporting 15.2% correct-and-secure for the best agent and 9.2% on average. Patir et al. [2025] automate the joint pipeline in DualGauge over 154 tasks and 10 models, observing secure-pass@1 below 12% while functional pass@1 exceeds 50%.

What we do here differs in domain, not in framing. The artifact is a single expression instead of a program. The vulnerability class is ReDoS instead of a CWE taxonomy. Correctness is partly decidable, not established by tests alone. And the NL-to-regex literature supplies a human reference for every task, which gives us a human baseline on the same instrument. That last point is what makes Section 4.4 possible. Reference-free benchmarks cannot run that comparison at all.

Our numbers turned out not to match theirs. Section 4.5 works through the difference.

Sampling-based metrics. We use the unbiased $PASS@k$ estimator of Chen et al. [2021], with the degeneracy noted in Section 3.4.

Serving-layer variance. Hosted inference routers may serve the same model slug from multiple providers at differing numerical precision. LessWrong [2025] report that 31 of 32 surveyed AI-safety codebases using such a router did not pin the serving provider, making their results a measurement of the router’s routing policy in addition to the model. Section 3.3 describes the discipline we adopt in response.

3 Method

3.1 Task and prompt

Each task supplies a natural-language description. The model returns one regular expression. The prompt is fixed across all models and all tasks:

system: You translate a natural-language description into a single Python **re**-compatible regular expression. Reply with ONLY the pattern in one fenced code block, no explanation.

user: *{task description, verbatim}*

Reply with only the regular expression pattern in a single fenced code block.

We do not tune this prompt. Prompt optimisation would raise all scores and destroy comparability with prior work on the same corpus. Section 6 notes the corresponding threat.

Extraction. We take the final fenced code block, else the whole trimmed reply, then strip at most one layer of host-language quoting (`r'...`, `'...'`, `"..."`, backticks, `/.../flags`). Scored literally, `r'\d+$'` is a pattern matching the letter `r`, and would fail for reasons unrelated to regex competence. This affected 7–18 of 1,350 responses per model, under 1.4% throughout, and changes no conclusion. We release both the normalised and the unnormalised scores.

3.2 Corpus

We use Re(gEx|DoS)Eval [S2E Lab — VERIFY, 2024], 762 tasks drawn from real user requests, each with a description, positive and negative test strings, and a human-written reference pattern. We evaluate 450 tasks, selected at uniform stride across the corpus rather than as a prefix (the corpus is difficulty-ordered at its head). The subset size was budget-determined.

Match semantics are set per corpus. Re(gEx|DoS)Eval references satisfy their own tests at 100% under search semantics and 94% under full-match. Choose wrong and 46 gold patterns are scored as failures. We verified the loaded configuration by scoring every reference against its own tests before the run (Section 5.1).

3.3 Models and serving discipline

Eleven models spanning frontier and open-weights systems across a 98× price range. Every request pins a single named provider with fallback disabled, so a request is either served by the named endpoint or fails visibly. Substitution failures did occur and are reported as failures rather than silently re-routed (Appendix A).

For open-weights models the pin includes quantisation where the router discloses it: GLM-5.2 and DeepSeek-V4-Flash are served at 4-bit by some providers and 8-bit by others, and we pin 8-bit. One pin went stale within an hour of being set (the provider ceased serving that model), so pins are re-verified immediately before each run.

The pin is a request, not a guarantee. So we record which provider actually served each response and check afterwards that it matches. All eleven models came back served entirely by the endpoint they were pinned to.

3.4 Metrics

Let $T = (d, P, N, r)$ be a task with description d , positive examples P , negative examples N and reference pattern r . Let p be a candidate.

Correctness.

$$\text{correct}(p, T) = [\forall x \in P : \text{match}(p, x)] \wedge [\forall y \in N : \neg \text{match}(p, y)]$$

evaluated with CPython’s **re** under the corpus’s declared semantics. This is *reference-independent*.

Semantic equivalence. Both p and r are compiled to finite automata and compared, yielding a verdict in $\{\text{EQ}, \text{DIFF}, \text{UNDEC}, \text{UNSUP}\}$. When the verdict is DIFF the engine returns a *witness*: a shortest string in the symmetric difference, which renders the judgment falsifiable by inspection.

Equivalence is decidable only on the regular subset. Backreferences make the language non-regular, and equivalence then has no algorithmic answer at all. Those cases return UNDEC . Between 82 and 133 of 450 tasks per model land there. We therefore report two figures:

$$\text{DFA-EQ} = \Pr[\text{EQ}], \quad \text{DFA-EQ}_{\text{dec}} = \Pr[\text{EQ} \mid \text{verdict} \neq \text{UNDEC}].$$

The first counts undecidable comparisons as failures. It is a lower bound and cannot flatter. The second isolates the model from the engine’s reach. Report only the second and you inflate. Report only the first and you charge the model for a theorem. This metric is *reference-dependent*, which Section 5.1 shows to be decisive.

ReDoS safety. $\text{vuln}(p) = [\text{screen}(p) \neq \text{SAFE}]$, where `screen` performs structural analysis for known-vulnerable shapes followed by an empirical pass against attack strings under timeout. The structural pass covers three of the five families catalogued by ICPC 2024 authors — `VERIFY` [2024]. So `VULNERABLE` is a *lower bound*, and `SAFE` denotes a screening outcome, not a proof. This metric is *reference-independent*.

Composite.

$$\text{usable}(p, T) = \text{correct}(p, T) \wedge \neg \text{vuln}(p) \wedge [\text{verdict}(p, r) \neq \text{DIFF}]$$

The third conjunct excludes only *proven* difference. Undecidable and unsupported verdicts do not disqualify a pattern. `USABLE` is reference-dependent through that conjunct, and inherits the error characterised in Section 5.1.

Sampling. With n samples per task of which c satisfy a predicate, we use the unbiased estimator of Chen et al. [2021]:

$$\widehat{\text{metric@}k} = \mathbb{E}_T \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right].$$

We draw $n = 3$ and report $k = 3$. Note that at $n = k$ this estimator degenerates to $\mathbf{1}[c \geq 1]$ and buys no variance reduction over the naive any-of- n statistic. We report it as `@3` for comparability, not because the estimator is doing any work at this setting. $n > k$ would be required for the estimator’s variance properties to apply, and is left to future work with a larger budget.

3.5 Sampling configuration

Reasoning disabled. Four of the eleven models emit no hidden reasoning tokens. The other seven do. Evaluating them together with reasoning enabled would confound model identity with inference-time compute budget. All requests therefore disable reasoning, and reasoning-token counts are recorded per response to verify compliance (observed: zero throughout).

Temperature unset. We set `require_parameters=true`, which restricts routing to providers that honour submitted sampling parameters rather than silently discarding them. Six of the eleven models reject `temperature` outright. Together, the two settings make the request unroutable. We therefore omit `temperature` entirely and each model samples at its provider default. That keeps

the non-silent-discard guarantee and costs us controlled sampling. We think it is the right trade and note it as a limitation.

Since $k > 1$ is only informative under output diversity, we verified diversity directly: repeated queries on a fixed task yielded 3, 3, 3 and 2 distinct patterns for `claude-opus-5`, `gpt-5.6-sol`, `kimi-k3` and `glm-5.2` respectively. The analysis pipeline additionally warns if $> 90\%$ of a model’s tasks return identical samples.

3.6 Controls

Three synthetic responses go through the identical scoring path on every run. The task’s own reference must come back correct and usable. The pattern `z{5}` must fail every correctness check. `(a+)+b` must be flagged vulnerable. If any control misbehaves we throw the run away instead of reporting it.

The failure this catches is a scorer that quietly returns nulls, which looks exactly like every model performing badly. All controls behaved on all eleven models.

4 Results

4.1 Main results

Model	PASS@3	USABLE@3	VULNERABLE@3	DFA-EQ@3	DFA-EQ _{dec}	EXACT@3	undec.	fail
kimi-k3	47.2	24.8	14.1	15.2	17.5	6.3	82	16
claude-opus-5	47.5	23.0	15.5	14.0	17.0	5.4	109	36
qwen3.6-max-preview	42.4	21.6	9.1	13.8	17.4	3.8	94	0
gpt-5.6-sol	42.2	21.1	10.2	9.6	13.6	2.0	133	1
deepseek-v4-flash-0731	38.0	19.8	12.0	11.3	14.3	3.3	93	0
qwen3.6-plus	39.8	19.8	9.8	11.6	14.8	3.8	99	0
glm-5.2	42.4	18.7	14.2	10.4	12.9	3.8	86	0
gpt-5.6-luna	39.3	18.7	11.8	9.3	12.2	2.0	113	1
gpt-5.6-terra	42.2	18.7	12.0	10.2	13.1	2.0	100	0
claude-sonnet-5	40.7	18.0	10.9	10.4	13.3	3.3	97	0
gemini-3.1-flash-lite	38.7	17.1	12.0	9.3	11.8	3.1	95	0

Table 1: Primary results, 450 tasks, $n = k = 3$. All figures percentages except *undec.* (count of tasks with undecidable equivalence, of 450) and *fail* (count of failed requests, of 1,350). Ordering is by USABLE@3 and should be read as a banding, not a ranking (Section 5.3).

The apparent headline in Table 1 is the gap between PASS@3 (38.0–47.5%) and USABLE@3 (17.1–24.8%): approximately half of all test-passing generations fail at least one of the remaining two criteria, and the ratio is stable across the entire price range.

Section 4.2 shows that this reading is wrong, or at least that it attributes the gap to the wrong cause. We present it here as stated because it is what the composite reports, and because constructing such a composite is a natural thing to do. The point of the next section is that building one without decomposing it was a mistake, and it was ours.

4.2 Decomposing the gap

USABLE conjoins three conditions. Table 2 attributes the drop from PASS@3 to USABLE@3 between the safety term and the equivalence term, and additionally reports the rate of ReDoS vulnerability *conditional on functional correctness*.

Model	PASS@3	−safety	C&S@3	−equiv.	USABLE@3	vuln. correct
kimi-k3	46.5	2.5	44.0	20.2	23.8	5.6
claude-opus-5	46.1	4.9	41.2	20.4	20.8	10.0
qwen3.6-max-preview	42.4	2.7	39.8	18.2	21.6	7.0
gpt-5.6-sol	42.1	1.6	40.5	19.6	20.9	5.6
deepseek-v4-flash-0731	38.0	1.8	36.2	16.4	19.8	5.7
qwen3.6-plus	39.8	3.3	36.4	16.7	19.8	8.1
glm-5.2	42.4	4.0	38.4	19.8	18.7	9.3
gpt-5.6-luna	39.2	2.9	36.3	17.8	18.5	8.1
gpt-5.6-terra	42.2	2.9	39.3	20.7	18.7	7.3
claude-sonnet-5	40.7	2.7	38.0	20.0	18.0	6.0
gemini-3.1-flash-lite	38.7	2.9	35.8	18.7	17.1	8.3
<i>mean</i>	—	2.9	—	18.9	—	7.4

Table 2: Decomposition of the PASS→USABLE gap. “C&S” is correct-and-secure: correctness conjoined with safety, omitting the equivalence term. The final column is the share of functionally correct patterns that are ReDoS-vulnerable.

Safety costs 2.9 percentage points on average. Equivalence costs 18.9. So the equivalence term supplies **87%** of the gap that made the composite look interesting.

That would be unremarkable if the equivalence term were sound. It is not. Section 5.1 shows, from an independent audit, that roughly 85% of what it counts against a model is reference-set error or an underspecified prompt. Our composite was mostly reporting its own weakest component back to us.

We did not see this coming. The decomposition was run late, after the audit and after the first draft, because a reviewer of an earlier version asked how our numbers compared to joint benchmarks in other domains. Answering that required computing correct-and-secure without the equivalence term, and the answer changed the paper.

Restricting attention to the two reference-independent criteria, the finding is far more modest and far better supported: **7.4% of functionally correct regular expressions are ReDoS-vulnerable** (range 5.6–10.0%).

We count 135 model-task pairs in which a pattern satisfied every test and was simultaneously screened as unsafe. As a rate over correct generations instead of a raw count, that is the 7.4% above. About one in fourteen, not one in two.

EXACT@3, string identity with the reference, runs from 2.0% to 6.3%. String-comparison scoring would therefore misrank essentially every system, which is what justifies the automata-theoretic treatment.

4.3 Sample budget and the fragility of ordering

Because $n = k = 3$ renders the PASS@ k estimator degenerate (Section 3.4), we additionally report @1, which is the per-sample success rate and the quantity most directly comparable to single-sample protocols. Table 3 gives both.

The comparison is instructive beyond bookkeeping. At @3, kimi-k3 attains the highest USABLE at 23.8%. At @1 the leader is claude-opus-5 on 17.3%, and kimi-k3 drops to joint third. *The induced ordering depends on the sample budget.* Systems differ not only in per-attempt quality but in how much they benefit from additional attempts, and a leaderboard that fixes k silently fixes a weighting between those two properties. This is a further reason to prefer banded reporting (Section 5.3), and a reason to state k prominently whenever an @ k metric is ranked.

Model	PASS@1	PASS@3	USABLE@1	USABLE@3	VULNERABLE@1	$n < 3$
kimi-k3	35.6	46.5	16.0	23.8	9.2	6
claude-opus-5	41.4	46.1	17.3	20.8	10.7	12
qwen3.6-max-preview	37.0	42.4	16.5	21.6	7.4	0
gpt-5.6-sol	37.3	42.1	16.0	20.9	8.4	1
deepseek-v4-flash-0731	30.0	38.0	14.4	19.8	7.7	0
qwen3.6-plus	35.5	39.8	15.9	19.8	8.0	0
glm-5.2	34.1	42.4	13.5	18.7	9.6	0
gpt-5.6-luna	33.8	39.2	15.2	18.5	8.7	1
gpt-5.6-terra	36.6	42.2	14.8	18.7	8.9	0
claude-sonnet-5	36.8	40.7	15.3	18.0	9.3	0
gemini-3.1-flash-lite	33.2	38.7	13.9	17.1	9.5	0

Table 3: Per-sample (@1) against any-of-three (@3) estimates, computed from per-sample success counts. The final column gives the number of tasks with fewer than three usable samples, which are excluded from that model’s @3 estimate. That is why @3 here differs slightly from Table 1, which is computed over all answered tasks.

4.4 ReDoS and the human baseline

Table 4 applies the identical safety screen to the corpus’s 450 human-written reference patterns. For comparability with the human set (one pattern per task) we score one sample per task per model rather than the any-of-three statistic of Table 1.

Source	n	vulnerable (%)	exponential (%)	polynomial (%)
<i>Human reference answers</i>	450	13.6	6.4	7.1
kimi-k3	447	10.7	5.4	5.4
claude-opus-5	444	10.6	7.4	3.2
glm-5.2	450	10.2	5.8	4.4
gemini-3.1-flash-lite	450	9.8	5.1	4.7
gpt-5.6-sol	450	9.3	5.1	4.2
claude-sonnet-5	450	8.9	6.0	2.9
gpt-5.6-terra	450	8.7	4.9	3.8
qwen3.6-plus	450	8.4	5.1	3.3
gpt-5.6-luna	450	8.0	4.0	4.0
deepseek-v4-flash-0731	450	7.3	4.4	2.9
qwen3.6-max-preview	450	7.3	4.7	2.7
<i>All models pooled</i>	4941	9.0	5.3	3.8

Table 4: ReDoS screening of model outputs against the corpus’s own human-written reference patterns, one pattern per task in all cases.

Every model is safer than the reference set. This inverts the expected framing. Generation does not introduce ReDoS vulnerability relative to human practice. ReDoS vulnerability is endemic to human-written regular expressions, at a rate above what these models reproduce.

We regard this as the paper’s most consequential empirical finding, because it changes the implied remedy. A model-deficiency framing implies waiting for better models. A distributional framing implies that safety screening must be applied at the point of use, since neither the generator nor the human corpus it was trained on performs that check.

Relation to prior work. ICPC 2024 authors — VERIFY [2024] report that LLM-generated regexes skew toward polynomial ReDoS. We do not reproduce this cleanly: pooled across models we observe 5.3% exponential against 3.8% polynomial, the opposite direction, while the human references skew polynomial (6.4% vs. 7.1%). The subcategory counts are small, so we make no strong claim. The discrepancy is worth a replication.

4.5 Comparison against joint benchmarks in other domains

Because USABLE carries a semantic-equivalence conjunct that the joint benchmarks of Section 2.1 do not, comparing it directly to their figures would understate us. We therefore also report *correct-and-secure*: correctness conjoined with safety, omitting the equivalence term, which is the construction those benchmarks use.

Benchmark	tasks	functional (%)	joint (%)	vuln. correct (%)
This work (regex, ReDoS)	450	42	39	7
BaxBench [Vero et al., 2025]	392	62 (best)	—	≈50
SecureAgentBench [Chen et al., 2025]	105	—	15.2 (best), 9.2 (mean)	—
DualGauge [Patir et al., 2025]	154	>50	<12	—

Table 5: Our correct-and-secure rate against joint correctness-security results from other domains. Task types, vulnerability classes and evaluation methods all differ substantially. The comparison is one of magnitude and of the ratio between functional and joint success, not of models or difficulty.

We do not reproduce the pattern. Vero et al. [2025] report that roughly half of functionally correct backends are exploitable. Chen et al. [2025] report 15.2% correct-and-secure for their strongest agent, against substantially higher functional correctness. Patir et al. [2025] report secure-pass@1 below 12% while functional pass@1 exceeds 50%. In each case the security criterion removes a large majority of functionally correct output.

In our setting it removes 7.4%.

We think the divergence is more useful than agreement would have been. Three explanations seem plausible. We cannot separate them with this data:

1. **Attack surface.** A regular expression is a single expression with one failure mode of interest. A backend application has many independently exploitable components, so the probability that at least one is vulnerable compounds with program size.
2. **Vulnerability class.** ReDoS is a structural property of the pattern, plausibly well represented in training data as a recognised anti-pattern. CWE-classified defects in application code are more diverse and more context-dependent.
3. **Task difficulty ceiling.** Our functional pass rate is itself low (38–47%), so the correct subset may be biased toward simpler tasks whose solutions have less structural room for catastrophic backtracking.

So the claim that generalises is weaker than “correct code is often insecure,” and more useful. *The size of the correctness-to-security penalty depends on the domain and has to be measured there.* Assuming it transfers is what we did, and we were wrong. Worse, our composite showed a gap of about the expected size for entirely the wrong reason (Section 4.2), which is exactly the situation in which an undecomposed joint metric will mislead you.

4.6 Cost

Model	USABLE@3 (%)	cost/task (\$ $\times 10^{-6}$)	total (\$)
deepseek-v4-flash-0731	19.8	25.6	0.03
gpt-5.6-luna	18.7	42.9	0.06
gemini-3.1-flash-lite	17.1	90.2	0.12
qwen3.6-plus	19.8	120.6	0.16
glm-5.2	18.7	158.2	0.21
qwen3.6-max-preview	21.6	387.9	0.52
gpt-5.6-terra	18.7	406.2	0.55
claude-sonnet-5	18.0	932.4	1.26
kimi-k3	24.8	1328.1	1.79
gpt-5.6-sol	21.1	2107.6	2.85
claude-opus-5	23.0	2514.2	3.39

Table 6: Cost per task at $n = 3$, measured from per-request billing returned by the serving layer rather than estimated from list prices.

The extremes differ by a factor of 98 in price and about three percentage points in USABLE@3, which is at the boundary of what we can resolve (Section 5.2). This observation is about this task only. It is not a general claim about model capability.

4.7 Refusals

A content filter blocked `claude-opus-5` on 29 requests, 2.1% of its calls, spread across five tasks. Four have some security flavour: strings that exclude a single quotation mark, a six-character alphanumeric password, hex byte sequences, and SQL update and insert statements. The fifth is “*Matches a file extention.*” No other model refused anything.

The refusals were not deterministic. Several tasks came back refused on one sample and answered on the other two, under identical settings. You can only see that at $k > 1$. A single-sample protocol would have logged a flat failure and moved on. We count refusals as their own failure category, not as wrong answers (Appendix A).

4.8 Inference-time compute

On the corpus’s simplest task (“*Matches exactly 1 numeric digit (0-9).*”), `qwen3.6-max-preview` emitted 1,571 hidden reasoning tokens before answering `^[0-9]$`, at a cost of \$0.0098. With reasoning disabled it produced the identical answer in 10 tokens at 1/100th the cost. Asking that model for *low* reasoning effort produced 2,375 reasoning tokens, more than the default. Effort controls are not reliably monotone.

A reasoning-enabled comparison on 12 tasks across five models yielded per-request costs $15.7\times$ higher and no interpretable accuracy signal at that sample size (± 14 percentage points per cell). We report the cost result and explicitly decline to report an accuracy conclusion.

5 Measurement Validity

5.1 The reference standard contains errors

First we ruled out a loading fault. 449 of 450 reference patterns satisfy their own tests, so the corpus semantics and dialect are configured correctly. The real defect is quieter. A reference can pass its own tests and still fail to denote what its description says, whenever the tests never exercise the difference.

We drew a seeded random sample of 60 cases in which a model *passed every test* yet received a DIFF verdict, and adjudicated 14 in detail.

Adjudicated cause	count	share
Reference pattern is incorrect	5	36%
Description underspecifies the disputed property	6	43%
Model output is incorrect	3	21%

Table 7: Adjudication of 14 sampled DIFF verdicts on test-passing outputs. Per-case reasoning is released.

Separately, 19 of the 60 sampled disagreements (32%) differ only on non-ASCII input. Python’s `\d` is Unicode-aware and `[0-9]` is not, so the two are formally distinct languages and practically the same thing. Put both effects together and the model is clearly at fault in roughly **15%** of DIFF verdicts.

Representative cases:

- **Description:** “*It just accepts only positive numbers.*” The reference `^\d+([. ,]?\d+)?$` admits 0. The model excluded zero and was scored incorrect.
- **Description:** “*A very simple ISBN validation expression—it just checks for a 10 digit number.*” The reference is `^\d{9}[\d|X]$`. The character class contains digit, *pipe* and *X*: an alternation operator written inside a class, where it denotes a literal. The reference therefore accepts `000000000|`, which is the witness separating it from the model’s `^\d{9}[\dX]$`.

- **Description:** “*...make sure commas are in the rite place (if present).*” The reference

`^\$?\d{1,3}(,?\d{3})*(\.\d{1,2})?$`

makes the comma optional, admitting `0,000000` and negating the stated purpose. The model enforced comma placement and was scored incorrect.

- **Underspecification:** “*Matches any single upper- or lower-case letter.*” The reference `^[a-zA-Z]$` requires a whole-string match. A candidate `[A-Za-z]` requires only occurrence. The description does not distinguish the two. We tested whether anchoring alone explains failures generally: it accounts for approximately 6%, so the effect is diffuse rather than a single correctable convention.

Consequence. DFA-EQ and USABLE are lower bounds on model correctness by an unmodelled margin. PASS and VULNERABLE are unaffected, since neither consults the reference.

Limitation of this analysis. Fourteen cases, judged by us, on our own benchmark. That is a weak basis for a precise number and we do not want it cited as one. We release the reasoning for each case, and the unadjudicated cases keep an empty verdict field so someone else can fill them in. Anyone quoting 15% should re-derive it from a larger sample with annotators who have no stake in the answer. We report it because the direction is not in doubt, and because the direction is what changes how the composite should be read.

5.2 Paired inference

Our first analysis compared systems with independent binomial confidence intervals. That is the wrong estimator for this design. All eleven models answer the same items, so task difficulty is a shared random effect. Treating the systems as independent samples charges every one of them separately for the same variation.

A paired bootstrap fixes this. Resample tasks with replacement, score every model on each resample, and form the per-pair differences. Task difficulty cancels, and only genuine disagreement contributes. With $B = 10,000$ resamples and a fixed seed:

Procedure	pairs resolved at 95% (of 55)
Independent binomial intervals	1
Paired bootstrap, <code>USABLE@3</code>	9
Paired bootstrap, <code>PASS@3</code>	15
Paired bootstrap, <code>VULNERABLE@3</code>	15

Table 8: Resolvable pairwise comparisons under independent versus paired inference on identical data.

The correction is not marginal. It changes the number of defensible comparisons by nearly an order of magnitude. We flag it because independent intervals look like the default in adjacent evaluation work, and the design they are applied to, all systems answering all items, is close to universal.

Even corrected, the middle of the table does not separate. For example `qwen3.6-max-preview` versus `gpt-5.6-sol` yields +0.7% with a 95% interval of $[-2.9\%, +4.3\%]$.

5.3 Discriminative power of the corpus

There is a structural reason behind this. On `USABLE@3`, **62% of tasks produce the same outcome for all eleven models**. On `PASS@3` it is 56%, on `VULNERABLE@3`, 77%. Only about 167 of 450 tasks carry any discriminative signal at all.

For calibration: resolving a 3-percentage-point difference at $p \approx 0.2$ with independent intervals requires roughly 2,800 items—more than the full 762-task corpus contains. Paired inference reduces this requirement substantially but cannot manufacture signal from items on which every system agrees. Benchmark construction for system comparison should therefore optimise for discriminative yield, not corpus size.

6 Threats to Validity

Contamination. The corpus predates every model we evaluated and is public. Some of what we measured may be memorisation. We have no held-out private task set, so we cannot put a bound on it. This is the largest unaddressed threat in the paper.

Single prompt. Every result here is conditional on one unoptimised prompt. Section 5.1 shows how often the task descriptions leave the anchoring convention unstated, so sensitivity to phrasing is plausibly large. We did not measure it.

Sampling control. We do not set temperature (Section 3.4), so it varies by provider default. We checked that outputs still vary across samples. We did not equalise the sampling.

Estimator degeneracy. At $n = k = 3$ the $\text{PASS}@k$ estimator reduces to any-of-3.

Coverage asymmetry. Nine models cover all 450 tasks. `kimik3` covers 447, having exhausted our budget mid-run, and `claude-opus-5` covers 445 because of refusals. The denominators differ by under 1%.

Screening, not proof. `VULNERABLE` is a lower bound (Section 3.4).

Single corpus. We do not test whether the observed banding survives corpus change.

Self-adjudication. See Section 5.1.

7 Recommendations for Evaluation Design

1. **Partition metrics by reference dependence, and decompose any composite that spans the partition.** Metrics evaluated by execution against inputs (`PASS`, `VULNERABLE`) are robust to reference-set defects. Metrics evaluated against a human answer key inherit its error rate. A composite conjoining both can report a large gap that is almost entirely attributable to the unreliable term—in our case 87% of it (Section 4.2)—while appearing to be a statement about the reliable ones.
2. **Sample and adjudicate disagreements.** It costs hours. In our case it changed the conclusion.
3. **Use paired inference when all systems answer all items.** Table 8.
4. **Report discriminative yield**, not only corpus size (Section 5.3).
5. **Pin the serving provider and verify post hoc.** Record what served each request, not only what was requested (Section 3.3).
6. **Instrument the scorer with controls** that must pass and must fail (Section 3.4).
7. **Release raw generations.** Anyone can then overturn a scoring decision without buying the inference again. That is the difference between a result someone can check and one they have to take on faith.

8 Conclusion

We set out to measure a gap between correct and shippable regular expressions. We built a composite metric, it showed a large gap, and decomposition showed that 87% of that gap came from the one term we had already shown to be unreliable. What survives is much smaller. 7.4% of functionally correct patterns carry a ReDoS vulnerability, against roughly 50% of functionally correct backends in comparable work. The correctness-to-security penalty depends on the domain, and in ours it is small.

Two findings survive that correction intact, both resting only on criteria that never consult a human answer key. Models are uniformly safer than the human-written reference patterns against which they are scored, which locates ReDoS prevalence in the training distribution rather than in generation. And the entire eleven-model field lies within eight percentage points across a $98\times$ range in inference cost.

The methodological result is the one we did not plan on. A metric that compares against a human answer key inherits that key’s error rate, and a composite built on such a term can produce a finding that is large, plausible and mostly spurious. We only found this by reading our own output after collecting it. The audit cost a few hours against a run that cost \$10.95, and it changed what we were willing to publish. We would do it earlier next time.

Acknowledgements

Scoring is performed by `regexbench` [Foothills Labs, 2026], prior work by the present authors, used here as a pinned dependency and not contributed by this paper. The corpus is Re(gEx|DoS)Eval [S2E Lab — VERIFY, 2024], used under its original terms and not redistributed.

References

- Junkai Chen, Huihui Huang, Yunbo Lyu, Junwen An, Jieke Shi, Chengran Yang, Ting Zhang, Haoye Tian, Yikun Li, Zhenhao Li, Xin Zhou, Xing Hu, and David Lo. SecureAgentBench: Benchmarking secure code generation under realistic vulnerability scenarios, 2025. 105 repository-level tasks. Best agent (SWE-agent with DeepSeek-V3.1) attains 15.2% correct-and-secure; mean across agents 9.2%.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. Source of the unbiased pass@k estimator used in Section 3.4.
- James C. Davis, Christy A. Coghlan, Francisco Servant, and Dongyoon Lee. The impact of regular expression denial of service (redos) in practice: An empirical study at the ecosystem scale. In *Proceedings of ESEC/FSE*, 2018.
- James C. Davis, Louis G. Michael IV, Christy A. Coghlan, Francisco Servant, and Dongyoon Lee. Why aren’t regular expressions a lingua franca? an empirical study on the re-use and portability of regular expressions. In *Proceedings of ESEC/FSE*, 2019. Artifact: <https://github.com/VTLeeLab/LinguaFranca-FSE19>.
- Foothills Labs. regexbench: Evaluate generated regular expressions — semantic equivalence, correctness, and redos safety. <https://github.com/foothills-labs/regexbench>, 2026. Version

0.4.0, pinned at commit 05d7547b. Apache-2.0. Prior work by the present authors; used here as a dependency, not contributed by this paper.

ICPC 2024 authors — VERIFY. On the security of regular expressions generated by large language models. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension (ICPC)*, 2024. doi: 10.1145/3643916.3644424. Taxonomy of five ReDoS vulnerability families in LLM-generated patterns; reports a skew toward polynomial ReDoS.

Nate Kushman and Regina Barzilay. Using semantic unification to generate regular expressions from natural language. In *Proceedings of NAACL-HLT*, 2013. The KB13 corpus.

LessWrong. Not pinning your openrouter provider might invalidate your research. <https://www.lesswrong.com/posts/KsyoSAYBRXtwzSugg/>, 2025. Reports that 31 of 32 surveyed AI-safety codebases using OpenRouter did so without pinning the serving provider.

Nicholas Locascio, Karthik Narasimhan, Eduardo DeLeon, Nate Kushman, and Regina Barzilay. Neural generation of regular expressions from natural language with minimal domain knowledge. In *Proceedings of EMNLP*, 2016. Introduces NL-RX; establishes DFA-equivalence as the standard correctness criterion for this task.

Rupam Patir, Keyan Guo, Suvadra Barua, Abhijeet Pathak, Dinesh Gudimetla, Jiawei Guo, Hongxin Hu, and Haipeng Cai. DualGauge: Automated joint security-functionality benchmarking of specification-only code generation by LLMs and coding agents, 2025. 154 tasks, 10 models. Reports secure-pass@1 below 12% for all models evaluated while functional pass@1 exceeds 50%.

Jinjun Peng, Leyi Cui, Kele Huang, Junfeng Yang, and Baishakhi Ray. CWEval: Outcome-driven evaluation on functionality and security of LLM code generation. In *Proceedings of the 2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*, 2025. 119 security-critical tasks over 31 CWEs in 5 languages, scored with outcome-driven oracles for functionality and security jointly. Artifact: <https://github.com/Co1lin/CWEval>.

S2E Lab — VERIFY. Re(gex|dos)eval: A benchmark for regular expression generation and redos detection. <https://github.com/s2e-lab/RegexEval>, 2024. 762 natural-language-to-regex tasks with tests and reference patterns. Corpus used in this work.

Mark Vero, Niels Mündler, Victor Chibotaru, Veselin Raychev, Maximilian Baader, Nikola Jovanović, Jingxuan He, and Martin Vechev. BaxBench: Can LLMs generate correct and secure backends? In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, 2025. 392 backend tasks (28 scenarios $\times$ 14 frameworks, 6 languages). Reports that roughly half of functionally correct solutions are exploitable.

A Failure taxonomy

Of 14,850 requests, 54 (0.36%) failed and are reported rather than excluded.

Empty completions are classified by reported `finish_reason` into refusal, token-budget exhaustion, and unexplained, rather than being scored as empty patterns—which would be indistinguishable from a maximally poor answer.

Model	Cause	Count
claude-opus-5	content-filter refusal	29
kimi-k3	account spending limit reached mid-run	11
kimi-k3	response without resolved provider	3
gpt-5.6-sol	response without resolved provider	1

Table 9: Request failures by cause. Responses lacking a resolved provider are discarded rather than scored, as provenance is a precondition for reproducibility.

B Operational hazards

We document configuration hazards encountered during this study, each of which produced plausible-looking but invalid data. We believe this list has independent value.

1. **Parameter combinations that are silently unroutable.** Setting `temperature` together with `require_parameters=true` yields a hard routing failure on 6 of 11 models. Our first pilot failed all 396 calls.
2. **Systems that cannot satisfy the experimental condition.** Two Gemini variants return *“Reasoning is mandatory for this endpoint”* and cannot be evaluated with reasoning disabled. Undetected, this would have placed one reasoning-enabled system in an otherwise reasoning-disabled comparison.
3. **Empty completions from budget exhaustion.** At a 200-token cap, reasoning models consumed the entire budget on hidden reasoning and returned no content.
4. **Resumption across configuration changes.** Our collection is resumable by (model, task, sample). An aborted run at one token cap left truncated rows that a resumed run at a different cap treated as complete, interleaving two configurations indistinguishably. Rows now carry a configuration fingerprint and resumption refuses on mismatch.
5. **Sample collapse in scoring.** An early scorer retained only the final sample per task, which would have rendered all @3 statistics disguised @1 statistics with no visible symptom.

C Reproduction

```
git clone https://github.com/foothills-labs/regexleaderboard
cd regexleaderboard
make setup # pinned scorer + corpus download
make score RUN=sweep
```

Scoring reads only the committed generations. No API access, no expenditure. We verified reproduction from a clean checkout with fresh dependency installation and corpus download, obtaining bit-identical metrics. A reduced form of this check executes in continuous integration on every commit.